

Contents

hide

Beginning

3. Introduction to Monte Carlo simulation

Average Energy and Magnetisation

TASK 3a:

Importance Sampling

Modifying IsingLattice.py

TASK 3b:

TASK 3c:

Programming a 2D Ising Model/Introduction to Monte Carlo simulation

Page

Discussion

Read

View source

View history

Tools

This is the third section of the Ising model experiment.

Main page: Programming a 2D Ising Model

Previous section: 2. Calculating the energy and magnetisation

Next section: 4. Accelerating the code

3. Introduction to Monte Carlo simulation

Average Energy and Magnetisation

Consider again the expressions for the average energy and magnetisation that we gave in the introduction.

$$\langle M \rangle_T = \frac{1}{Z} \sum_{\alpha} M(\alpha) \exp \left\{ -\frac{E(\alpha)}{k_B T} \right\}$$

$$\langle E \rangle_T = \frac{1}{Z} \sum_{\alpha} E(\alpha) \exp \left\{ -\frac{E(\alpha)}{k_B T} \right\}$$

Imagine we want to evaluate these at a particular temperature, in a system of 100 spins.

TASK 3a:

How many configurations are available to a system with 100 spins? To evaluate these expressions, we have to calculate the energy and magnetisation for each of these configurations, then perform the sum. Let's be very, very, generous, and say that we can analyse 1×10⁹ configurations per second with our computer. How long will it take to evaluate a single value of $\langle M \rangle_T$?

Clearly, we need to try a cleverer approach.

For the overwhelming majority of the possible configurations, the Boltzmann factor, $\exp \left\{ -\frac{E(\alpha)}{k_B T} \right\}$, will be very small, and that state

will contribute very little to the average value. We can save an enormous amount of time, and make this problem tractable, if we only consider the states whose Boltzmann factor is not so vanishingly small. This technique is called *importance sampling* — instead of sampling every point in phase space, we sample only those that the system is likely to occupy.

Importance Sampling

This is easily stated, of course, but how can we know which states contribute a lot to the average without actually calculating the energy? So far, we have been imagining generating random states of the system; that is to say, we have been choosing α from a uniform distribution in which no arrangement of spins is any more likely to be chosen than any other. If we instead had a method which generated states randomly from the probability distribution $\exp \left\{ -\frac{E(\alpha)}{k_B T} \right\}$, then our problem would be solved. The method

which allows us to do this was developed in the 1950s, and is called the [Metropolis Monte Carlo \(MMC\)](#) [↗] method, or more often simply "Monte Carlo simulation". This was one of the major breakthroughs of the early days of computational science, and in the year 2000, the IEEE named among the ["Top 10 Algorithms of the 20th Century"](#) [↗].

The algorithm is as follows:

- Start from a given configuration of spins, α_0 , with energy E_0 .
- Choose a single spin **at random**, and "flip" it, to generate a new configuration α_1 .
- Calculate the energy of this new configuration, E_1 .
- Calculate the energy difference between the states, $\Delta E = E_1 - E_0$.
 - If the $\Delta E \leq 0$ (the spin flipping decreased the energy), then we **accept** the new configuration.
 - We set $\alpha_0 = \alpha_1$, and $E_0 = E_1$, and then **go to step 5**.
 - If $\Delta E > 0$, the spin flipping increased the energy. By considering the probability of observing the starting and final states, α_0 and α_1 , it can be shown that the probability for the transition between the two to occur is $\exp \left\{ -\frac{\Delta E}{k_B T} \right\}$. To ensure that we only accept this kind of spin flip with the correct probability, we use the following procedure:
 - Choose a random number, R , in the interval [0,1).
 - If $R \leq \exp \left\{ -\frac{\Delta E}{k_B T} \right\}$, we **accept** the new configuration.
 - We set $\alpha_0 = \alpha_1$, and $E_0 = E_1$, and then **go to step 5**.
 - If $R > \exp \left\{ -\frac{\Delta E}{k_B T} \right\}$, we **reject** the new configuration.
 - α_0 and E_0 are left unchanged. **Go to step 5**.
- Update the running averages of the energy and magnetisation.
- Monte Carlo "step" complete, **return to step 2**.

Step 4 is the key to the method. By accepting moves which increase the energy only with a certain probability, we ensure that the sequence of states that we generate is correctly distributed. A transition which carries a very large energy penalty, ΔE is extremely unlikely to be selected. The use of random numbers in this step is the reason that the method acquired the name "Monte Carlo", after the casinos located there!

Note, unlike molecular dynamics, each step only moves a single atom. In Monte Carlo simulation, there is the notion of a cycle which is N_{spins} -steps, so that each spin on average has an attempted flip per cycle.

If you are interested in the mathematical details of why this procedure generates a sequence of states distributed in the correct way, consult the Monte Carlo chapter of ["Understanding Molecular Simulation"](#) [↗], by Frenkel and Smit, but a discussion of this is not required for the experiment. Chapter 7 of ["Statistical Mechanics: Theory and Molecular Simulation"](#) [↗] by Tuckerman also gives a good description.

Modifying IsingLattice.py

Our IsingLattice object contains a stub function, `montecarlostep(T)`, which takes a single argument — the temperature at which we want to do the cycle. There are also attributes used to calculate the average energy and magnetisation and respective variances: `self.E_tally`, `self.E2_tally`, `self.M_tally`, `self.M2_tally`, and `self.n_steps`. You should use these values to keep a running sum of all the energies, squared energies, magnetisations, and squared magnetisations that your system has sampled, as well as the number of steps that have been performed. Finally, there is also a function `statistics()`, that takes no arguments and should return the average energy, squared energy, magnetisation, squared magnetisation and current step, **in that order**.

TASK 3b:

Implement a single step of the above algorithm in the `montecarlostep(T)` function. This function should return the energy of your lattice and the magnetisation at the end of the cycle. Complete the `statistics()` function. This should return the following quantities whenever it is called: $\langle E \rangle$, $\langle E^2 \rangle$, $\langle M \rangle$, $\langle M^2 \rangle$, and N_{steps} , the number of Monte Carlo steps that have elapsed. Put your code for these functions in the report.

Note on units: In our simulation we will be using an unitless scaled energy $\tilde{E} = \frac{E}{J}$, and a unitless scaled temperature $\tilde{T} = \frac{k_B T}{J}$, where J is the interaction energy between spins discussed previously. The use of these scaled quantities in practice means that in the code we can set k_B to numerically equal to 1. Going forward we will drop the $\sim$ from the notation of energy and temperature.

You have been provided with a script, `lAnim.py`, which will use your `IsingLattice` object to run a Monte Carlo simulation and display the output on the screen in real time. By default, this simulation will run an 8×8 lattice at $\tilde{T}$ =1.0, which is below the critical temperature. You will see a representation of the lattice, and graphs of the energy and magnetisation **per spin**. When you close the window, the script will use your `statistics()` function to print the average energy and magnetisation from the simulation. Note, you should run this script from the command line.

TASK 3c:

If $\tilde{T} < \tilde{T}_C$, do you expect a spontaneous magnetisation (i.e. do you expect $\langle M \rangle \neq 0$)? When the state of the simulation appears to stop changing (when you have reached an equilibrium state), use the controls to export the output to an SVG or PNG file and attach this to your report. You should also include the output from your `statistics()` function.

This is the third section of the Ising model experiment.

Main page: Programming a 2D Ising Model

Previous section: 2. Calculating the energy and magnetisation

Next section: 4. Accelerating the code

Appearance

hide

Text

- ☒ Small
- ☐ Standard
- ☐ Large

Width

- ☒ Standard
- ☐ Wide

